



SQLite com sqflite

Desenvolvimento Móvel

Aula 20/36

Dart

Provider

Sqflite

Model

Conceito

sqflite é como ter um armário com gavetas organizadas dentro do seu app — cada tabela é uma gaveta, cada registro é uma pasta dentro dela. `openDatabase()` cria ou abre o banco, com `onCreate` para criar tabelas e `onUpgrade` para migrações (versões do banco). Operações CRUD: `db.insert()` retorna o id gerado, `db.query()` retorna `List<Map<String, dynamic>>`, `db.update()` e `db.delete()` aceitam condições `where`. `rawQuery` permite SQL customizado para consultas complexas (JOIN, GROUP BY). O modelo de dados precisa de métodos `toMap()` e `fromMap()` para converter entre Map e objeto Dart. `path_provider` fornece `getDatabasesPath()` para achar o diretório do app. Migrations controlam mudanças no schema: ao mudar a versão, `onUpgrade` recebe `oldVersion` e `newVersion` para ALTER TABLE ou recriar tabelas.

SQLite - tarefas

id	titulo	feito
1	Estudar Flutter	0
2	Fazer compras	1
3	Projeto final	0

INSERT / SELECT / UPDATE / DELETE



Atividade Prática

Migre o to-do app de SharedPreferences para sqflite: crie a tabela tarefas com colunas id (INTEGER PRIMARY KEY AUTOINCREMENT), titulo (TEXT NOT NULL), concluida (INTEGER DEFAULT 0), criadaEm (TEXT). Implemente TodoDatabaseHelper com metodos: createTable, insertTodo, getTodos, updateTodo, deleteTodo. Substitua TodoProvider para usar o helper. Teste inserir, marcar como concluida, editar e excluir com persistencia apos fechar e reabrir o app.



Pesquisa

Pesquise o pacote floor (ORM para SQLite no Flutter): como anotar entidades com @Entity e @primaryKey, criar DAOs com @Query e @insert, e usar o banco gerado automaticamente via build_runner. Compare a produtividade com sqflite puro.



Requisitos

Flutter SDK. Adicionar sqflite e path_provider ao pubspec.yaml.